

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

John Vitor da Silva Cunha

**Implementação de um Algoritmo genético para
escalonamento de tarefas em ambiente
heterogêneo multiprocessado com custo de
comunicação**

Uberlândia, Brasil

2024

UNIVERSIDADE FEDERAL DE UBERLÂNDIA

John Vitor da Silva Cunha

**Implementação de um Algoritmo genético para
escalonamento de tarefas em ambiente heterogêneo
multiprocessado com custo de comunicação**

Trabalho de conclusão de curso apresentado
à Faculdade de Computação da Universidade
Federal de Uberlândia, como parte dos requi-
sitos exigidos para a obtenção título de Ba-
charel em Ciência da Computação.

Orientador: Paulo Henrique Ribeiro Gabriel

Universidade Federal de Uberlândia – UFU

Faculdade de Computação

Bacharelado em Ciência da Computação

Uberlândia, Brasil

2024

Lista de ilustrações

Figura 1 – Fonte: Silva (2020)	8
Figura 2 – Fonte: Silva (2020)	9
Figura 3 – Fonte: Silva (2020)	10
Figura 4 – Fonte: Silva (2020)	10
Figura 5 – Fonte: Silva (2020)	11
Figura 6 – Fonte: Silva (2020)	12
Figura 7 – Fonte: Costa (2022)	14

Lista de tabelas

Sumário

1	INTRODUÇÃO	5
2	FUNDAMENTAÇÃO TEÓRICA	7
2.1	Algoritmos Genéticos	7
2.1.1	Avaliação do indivíduo	9
2.1.2	Seleção	9
2.1.2.1	Torneio	9
2.1.2.2	Roleta	10
2.1.3	Operadores genéticos	10
2.1.3.1	Cruzamento	11
2.1.3.2	Mutação	11
2.1.4	Reinserção	12
2.2	Problema do escalonamento de tarefas	12
3	TRABALHOS RELACIONADOS	15
4	MÉTODO	17
	REFERÊNCIAS	19

1 Introdução

A evolução tecnológica fez com que diversas áreas do conhecimento demandassem cada vez mais de soluções de alto desempenho computacional. Visando isto, este trabalho abordará o problema do escalonamento de tarefas em multiprocessadores. Neste tipo de problema, uma tarefa computacional (um programa, ou processo) é dividida em sub-tarefas que podem ser executadas paralelamente em diferentes nós computacionais para otimizar alguma métrica de desempenho do sistema. Estes nós computacionais podem ser os núcleos de um processador ou um computador interligado a uma rede de computadores, onde cada nó executará paralelamente uma sub-tarefa. Existem diversas abordagens em algoritmos de escalonamento de tarefas, este trabalho explorará os Algoritmos Genéticos (**AG**) para resolver este problema. Algoritmos Genéticos são classes de algoritmos inspirados na teoria da evolução das espécies de Charles Darwin, onde uma população de soluções (indivíduos) é iniciada e a cada rodada podem ser aplicados aos indivíduos operações genéticas que o modificam e ao final da rodada a nova população é reavaliada. A ideia de AGs é que bons indivíduos consigam se reproduzir e se manter durante mais tempo na população e que esta população ao longo do tempo convirja para uma solução do problema.

Este tema é bastante discutido na academia, mas também tem grande aplicação prática. Cada vez mais indústrias necessitam de computação distribuída e de alto desempenho ([SANTOS, 2021](#)). Além disso, com o aumento do número de dados, até usuários comuns podem necessitar de alto desempenho ao usar sistemas computacionais ([Gabriela Pederneiras, 2019](#)) incentivados principalmente pela nova onda de IA. Visando isso é importante que sistemas computacionais escalonem bem suas sub-tarefas a fim de reduzir o tempo de execução ou a ociosidade dos computadores e/ou núcleos.

Este projeto visa implementar um algoritmo genético para o escalonamento de tarefas paralelas executadas em processadores heterogêneos (que o poder computacional é diferente entre os nós do sistema) e comparar o resultado com alguns outros trabalhos. O trabalho de [Costa \(2022\)](#) avaliou vários algoritmos diferentes para buscar soluções para este problema e avaliou e comparou o resultado dos diversos experimentos, porém nenhum dos algoritmos utilizados usou a abordagem evolutiva. Por isto, este trabalho pretende implementar um Algoritmo Genético usando os conjuntos de dados utilizados por [Costa \(2022\)](#) e ao final comparar o resultado de seu artigo com o resultado do AG. O trabalho de [Silva \(2020\)](#) busca discutir as diferentes abordagens e representações de algoritmos genéticos para o problema de escalonamento de tarefas em multiprocessadores. Este trabalho é uma revisão sistemática da literatura, foram revisados trabalhos publicados entre 1990 e 2018 sobre o tema, registrando uma variedade grande de representações de cromossomos

e operadores genéticos, além das formas diferentes de codificação para estes algoritmos considerando o problema de escalonamento de tarefas. Este artigo será bastante útil como consulta no desenvolvimento deste trabalho. Já o artigo de Santos et al. (2023) discute os algoritmos genéticos focados na otimização multi objetivo para o problema de escalonamento de tarefas, onde mais de uma métrica é levada em consideração para o AG otimizar. Neste trabalho também foi discutido o AG de Omara e Arafa (2010), sendo um algoritmo mono-objetivo e pode servir de base para este projeto.

O objetivo deste trabalho é implementar um algoritmo genético para o problema de escalonamento de tarefas em multiprocessadores heterogêneos e avaliá-lo a partir de uma métrica comum em outros trabalhos, provavelmente o *makespan* (tempo decorrido desde a execução da primeira tarefa até a conclusão da última tarefa). Como o problema de escalonamento de tarefas em multiprocessadores é NP-difícil, então é natural pensarmos em técnicas de otimização que trabalham com várias soluções no espaço de busca, justificando o uso do AG para esta classe de problemas. É também objetivo deste trabalho comparar os resultados obtidos com os resultados de Costa (2022) para avaliar se o AG é adequado para este problema frente aos outros algoritmos implementados pelo trabalho citado.

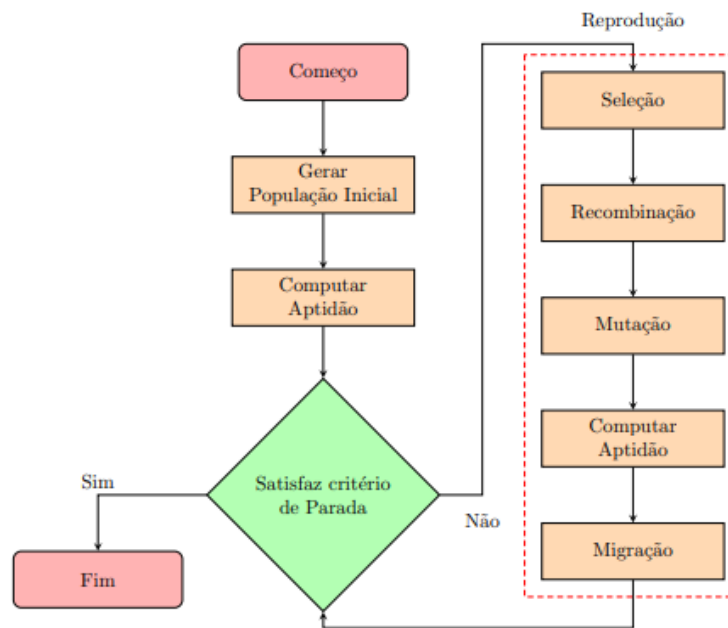
Para o desenvolvimento do trabalho será utilizada a linguagem de programação Python no ambiente do Google Colaboratory. Além disso, no trabalho de Costa (2022) foi proposto um algoritmo de conversão, que converte grafos STG, que representam as sub-tarefas e suas dependências em um ambiente homogêneo (poder computacional de igual capacidade), para o padrão DAG, que contempla mais de um processador e custo de comunicação entre as partes. Este algoritmo de conversão também será usado neste trabalho.

2 Fundamentação teórica

2.1 Algoritmos Genéticos

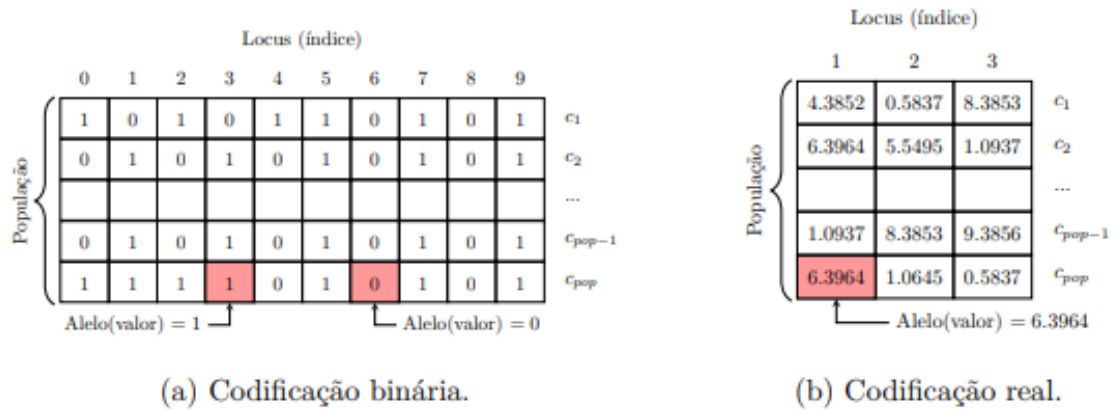
Algoritmos genéticos (**AG**) são classes de algoritmos inspirados na Teoria da Evolução de Charles Darwin. AGs utilizam técnicas de buscas inspirados no processo evolutivo da seleção natural, onde uma população de indivíduos é submetida a operadores genéticos inspirados pela reprodução sexual e assexual, visando produzir novos indivíduos e posteriormente selecionar indivíduos para permanecerem na população, levando em conta uma função de aptidão para avaliar o indivíduo. Esta técnica foi inicialmente proposta no trabalho de [Holland \(1975\)](#). Como dito anteriormente, os AGs operam uma população de indivíduos, onde cada indivíduo representa uma possível solução para o problema. Esta característica dos AGs permite ampliar o espaço de busca, diferente de outros algoritmos de busca que trabalham com apenas uma solução por vez, como a Busca de subida de encosta (*Hill climbing search*) e os algoritmos de busca mais clássicos como a Busca em profundidade (*Depth-First Search*) e a Busca em largura (*Breadth-First Search*). Portanto, AGs implementam uma busca paralela das soluções visando a sobrevivência dos indivíduos mais aptos.

No início do algoritmo, uma população inicial é gerada com os indivíduos sendo possíveis soluções do problema. Após a geração da população inicial, é calculado a aptidão de cada cromossomo usando a função de aptidão para o problema, a fim de classificar cada indivíduo com uma nota. Depois o algoritmo entra em um laço de repetição que terminará ao final de n gerações ou até alcançar algum critério de parada. Em cada geração, ns indivíduos são selecionados de acordo com sua nota de aptidão. Em seguida, os indivíduos selecionados passarão pelos operadores genéticos. O primeiro operador é o cruzamento (*crossover*), onde dois ou mais indivíduos (comumente chamados de pais) terão seus materiais genéticos recombinaados a fim de gerar um mais indivíduos novos (comumente chamados de filhos), por ser um algoritmo estocástico, o cruzamento tem a probabilidade ou não de ocorrer. Após a etapa de cruzamento, os indivíduos selecionados passarão para a etapa de mutação, que também tem a probabilidade de ocorrer ou não, nesta etapa os indivíduos sofrerão uma ou mais modificações aleatórias em seus genes. Depois do cruzamento e mutação, a população de indivíduos aumenta, o que pode levar a ser necessária uma atualização da população para a geração seguinte, pois a ideia dos AGs é sempre ter uma população de tamanho fixo (isso pode mudar a depender da natureza do problema). Essa atualização pode usar vários critérios para selecionar quais indivíduos da população permanecerão para a geração seguinte. Estes passos são repetidos até atingir n gerações ou algum critério de parada. Na figura 1 é ilustrado o funcionamento geral de

Figura 1 – Fonte: [Silva \(2020\)](#)

um AG.

Nos AGs, um cromossomo é a representação codificada de um indivíduo, onde o indivíduo pode ser visto como uma solução para o problema. A codificação de um indivíduo é um dos maiores desafios em um AG, pois é necessário abstrair o problema que o AG buscará resolver e essa abstração deve representar as características do problema além de ser compatível com os operadores do AG. Um mesmo problema pode comportar várias possibilidades de codificação dos cromossomos, porém diferentes representações podem levar o algoritmo a evoluir de diferentes formas. A codificação do cromossomo é muito importante para o desempenho do AG e diferentes codificações para um mesmo problema levam a um desempenho diferente do algoritmo, tanto na definição do espaço de busca quanto na codificação de indivíduos válidos, como mostrado por [Jelodar et al. \(2006\)](#). Uma população é um conjunto de indivíduos e conforme as gerações do algoritmo passam a população é evoluída pelos operadores genéticos, que podem criar novos indivíduos ou modificar indivíduos já existentes na população. A figura 2, mostra duas codificações diferentes para um mesmo problema. A primeira codificação representa o indivíduo como um vetor de valores binários, já a segunda, apresenta o indivíduo como um vetor de valores reais. Os alelos são os valores que cada posição do cromossomo pode assumir. Locus define o índice (ou posição) de um alelo no cromossomo. Chamamos de gene um alelo definido no locus do cromossomo e representa uma unidade individual de informação do indivíduo ([COELLO; LAMONT; VELDHUIZEN, 2007](#)).

Figura 2 – Fonte: [Silva \(2020\)](#)

2.1.1 Avaliação do indivíduo

Para avaliar a qualidade da solução de um indivíduo em um AG é utilizada uma função de aptidão, como dito na seção 1.1. O valor de aptidão (*fitness*) de um indivíduo mede sua qualidade e determina quão próximo ou distante o indivíduo está da solução do problema. A aptidão pode ser o custo da solução em problemas de otimização. Para problemas de minimização, quanto menor o valor de aptidão, melhor adaptado aquele indivíduo está e mais próximo ele está da solução global para o problema.

2.1.2 Seleção

A seleção é a etapa do AG onde indivíduos são selecionados para possíveis alterações genéticas, como a mutação e o cruzamento. Existem vários métodos de seleção de indivíduos para AGs e vários deles levam em conta a aptidão do indivíduo, há também a possibilidade da seleção totalmente aleatória, onde k indivíduos são selecionados aleatoriamente da população sem considerar seus valores de aptidão. Os indivíduos selecionados são chamados de pais, enquanto os novos indivíduos gerados pelas operações genéticas sobre os pais são chamados de filhos (ou descendentes). Os métodos de seleção mais popularmente utilizados em AGs são o método do torneio e o método da roleta.

2.1.2.1 Torneio

No método do torneio, n indivíduos são escolhidos aleatoriamente da população e têm seus valores de aptidão comparados, os indivíduos com os melhores valores de aptidão são selecionados para seguir para a fase de cruzamento e mutação ([GOLDBERG; DEB, 1991](#)). Na figura 3 é demonstrado o método de seleção por torneio.

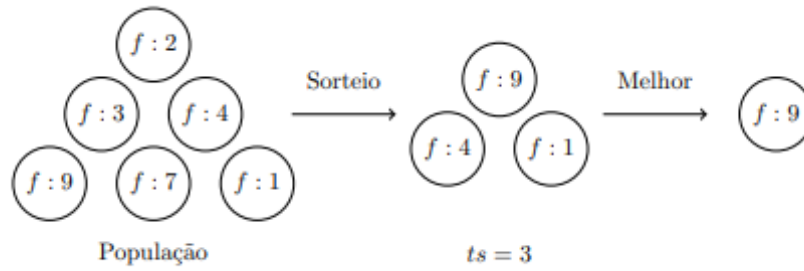


Figura 3 – Fonte: Silva (2020)

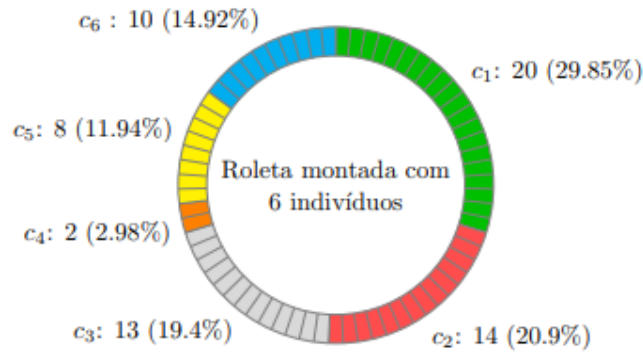


Figura 4 – Fonte: Silva (2020)

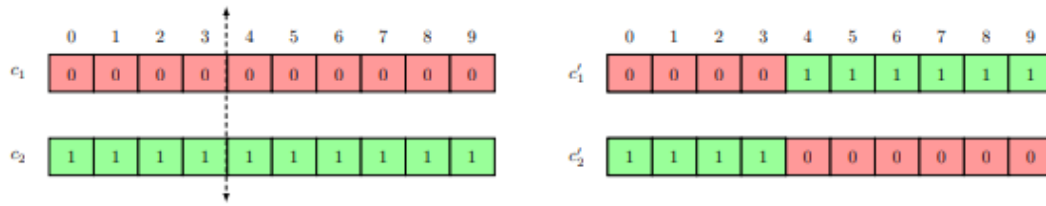
2.1.2.2 Roleta

A ideia do método da roleta é que os indivíduos com maior valor de aptidão tenham mais chances de serem selecionados, este método é definido como uma probabilidade cumulativa. A ideia principal é representar cada indivíduo da população como um intervalo proporcional a seu valor de aptidão. Após a construção dos intervalos para cada indivíduo, um número aleatório é gerado, respeitando o intervalo da roleta, após isso é selecionado o indivíduo onde seu intervalo na roleta contém o número gerado. No método da roleta todos os indivíduos têm chance de serem selecionados, porém, os de maior aptidão têm mais chances de serem sorteados, o que pode levar a uma perda de diversidade da população. A probabilidade de um indivíduo ser selecionado pelo método da roleta é dado pela expressão abaixo, onde f_i é a aptidão do i -ésimo indivíduo de uma população com N indivíduos:

$$ProbSelec = \frac{f_i}{\sum_i^n f_i}$$

2.1.3 Operadores genéticos

Uma das vantagens já ditas sobre os AGs é sua característica de trabalhar com várias soluções e de evoluir a população de soluções ao longo do tempo. Para explorar

Figura 5 – Fonte: [Silva \(2020\)](#)

ainda mais o espaço de busca à procura de outras soluções candidatas, os AGs utilizam os operadores genéticos, que, como dito na seção 1.1, são inspirados nos conceitos biológicos de reprodução sexuada e assexuada.

2.1.3.1 Cruzamento

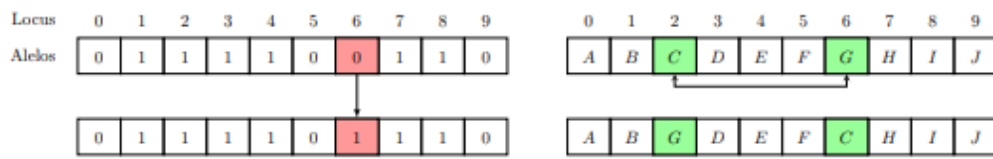
O cruzamento ou recombinação é baseado na reprodução sexuada, onde dois ou mais indivíduos são escolhidos como pais e têm seus materiais genéticos misturados a fim de criar um ou mais filhos. O cruzamento é um método estocástico, ou seja, a chance do cruzamento ocorrer e como será feita a recombinação depende de fatores aleatórios. Um dos métodos de cruzamento mais popular é o cruzamento de ponto único. No método do ponto único, um número aleatório de 0 até o tamanho do cromossomo é gerado e este número será o locus do ponto de corte que será feito nos pais a fim de gerar os filhos, os filhos então serão gerados pelo cruzamento do material genético de ambos os pais. No exemplo mais tradicional, utilizando o método do ponto único, dois pais podem gerar dois filhos, como demonstrado na figura 5 ([ZOMAYA; WARD; MACEY, 1999](#)).

2.1.3.2 Mutação

A mutação é baseada na reprodução assexuada, ou seja, é aplicada em um único indivíduo. A mutação é um operador unário, portando é uma modificação em um gene do cromossomo que pode modificar o indivíduo selecionado ou gerar um novo filho a partir da modificação do gene. Como o operador de cruzamento, a mutação também é um método estático e ocorre a depender de fatores aleatórios, o que causa uma mudança não enviesada ([EIBEN; SMITH, 2015](#)).

De acordo com [Golub e Kasapovic \(2002\)](#), a mutação é considerada um método para recuperar material genético perdido, permitindo uma exploração maior do espaço de busca e ajudando a prevenir a convergência prematura do algoritmo, aumentando sua diversidade. A mutação proposta por [Holland \(1975\)](#) é um operador que ocasionalmente muda os genes dos indivíduos, ocorrendo ou não mediante um número probabilístico $p_m \in [0, 1]$.

Dois exemplos de operadores de mutação são a mutação binária e mutação per-

Figura 6 – Fonte: [Silva \(2020\)](#)

mutada. Na mutação binária, um locus do cromossomo é sorteado aleatoriamente e tem seu gene modificado para um dos valores possíveis de alelo, método semelhante ao introduzido por [Holland \(1975\)](#). Já a mutação permutada é utilizada em cromossomos que não permitem a repetição de valores, neste caso, dois genes sorteados aleatoriamente tem seus alelos trocados, portanto, não gerando um indivíduo inválido. Na figura 6 há um exemplo de cada tipo de mutação explicado anteriormente.

2.1.4 Reinscrição

Ao aplicar os operadores genéticos, o número de indivíduos da população aumenta e é necessário escolher quais indivíduos serão passados para a próxima geração do AG e quais serão descartados. Esta etapa é chamada de reinscrição ou migração e há várias formas de execução.

Um dos métodos de reinscrição é o elitismo, onde uma porcentagem dos melhores indivíduos da geração atual serão passados diretamente para a geração seguinte, é um método bastante utilizado, pois permite que bons indivíduos permaneçam por mais tempo na população, porém se esta porcentagem for muito alta pode levar a uma convergência prematura do algoritmo. Há também a possibilidade de utilizar os métodos de seleção explicados na seção 1.1.3, que podem ajudar a manter uma boa taxa de diversidade da população, pois não descartam diretamente indivíduos com aptidão baixa. Um método mais agressivo de reinscrição é a reinscrição pura, onde toda a população será substituída pelos filhos gerados pelos operadores genéticos, porém neste método bons indivíduos pais podem ser substituídos por filhos piores, piorando o resultado do algoritmo.

2.2 Problema do escalonamento de tarefas

Com o avanço da computação de alto desempenho, viu-se a necessidade de utilizar arquiteturas com vários nós computacionais para resolver tarefas cada vez mais complexas ([BITTENCOURT et al., 2018](#)). Existem dois tipos de arquiteturas computacionais para resolver tarefas complexas, a arquitetura homogênea, onde os nós computacionais são idênticos, ou seja, uma tarefa executará da mesma forma em qualquer nó da arquitetura homogênea, exemplo deste tipo de arquitetura são os *clusters* computacionais. O outro

tipo de arquitetura, que será abordado neste trabalho, é a arquitetura heterogênea, onde os nós computacionais podem se diferir em velocidade de processamento, capacidade de armazenamento, consumo energético, entre outras características. Os nós computacionais podem ser processadores em um único computador ou computadores interligados entre si. Neste trabalho os nós computacionais estão interligados entre si, ou seja, qualquer nó da arquitetura pode se comunicar com qualquer outro nó desta arquitetura.

Do problema do escalonamento de tarefas surge a necessidade de organizar previamente quais sub-tarefas serão executadas paralelamente nos processadores, a fim de que a tarefa principal (o programa) seja executada no menor tempo possível. Em muitas destas tarefas é encontrada restrições que devem ser respeitadas, por exemplo, é possível que alguma sub-tarefa t_i dependa dos dados de uma tarefa t_j para ser executada, neste caso, a tarefa t_i só poderá ser executada quando a tarefa t_j terminar seu processamento. No exemplo anterior, se as duas tarefas, t_i e t_j , serão executadas em processadores diferentes, então é necessário computador o custo de comunicação para os dados de t_j chegarem ao processador que executará t_i . Se as duas tarefas serão executadas no mesmo processador, então o custo de comunicação é tido como zero (PADUA, 2011).

Uma das formas de representar um programa em sub-tarefas para serem executadas paralelamente é utilizando um acíclico orientado (DAG, do inglês *directed acyclic graph*). Um DAG é um grafo onde as arestas são orientadas e não há ciclos, ou seja, para qualquer vértice v deste grafo, não há nenhum caminho orientado que passe por v mais de uma vez. Cada vértice deste grafo é uma sub-tarefa que pode ser executada paralelamente com outras tarefas, as arestas servem para indicar a dependência de dados entre duas sub-tarefas e como dito anteriormente, um vértice v_i que tenha arestas chegando nele representa uma sub-tarefa que só poderá ser executada depois que todas as sub-tarefas que originam as arestas apontadas para v_i serem executadas.

Na figura 7 é apresentado um DAG de um programa com oito tarefas. O valor contido em cada vértice é o custo para processar aquela tarefa. Os pesos das arestas é tido como o custo de comunicação para os dados de uma tarefa t_i chegar a uma tarefa t_j quando executadas em diferentes processadores. Para o problema de escalonamento de tarefas heterogêneo, como cada processador tem diferentes capacidades de processamento, então é necessária uma formalização um pouco diferente do DAG acima, que considera as capacidades de processamento dos computadores iguais.

Para o problema de escalonamento de tarefas, é definido um conjunto de m processadores dado por $P = \{p_1, p_2, \dots, p_m\}$. De forma análoga temos o conjunto de tarefas definido por $T = \{t_1, t_2, \dots, t_m\}$. Para arquiteturas heterogêneas é definido uma matriz W de dimensões $n \times m$, onde cada elemento w_{ij} representa o custo de execução da tarefa t_i quando executada no processador p_j . Para definir o custo de comunicação entre os diferentes processadores é criada uma matriz B com dimensões $m \times m$, onde cada elemento

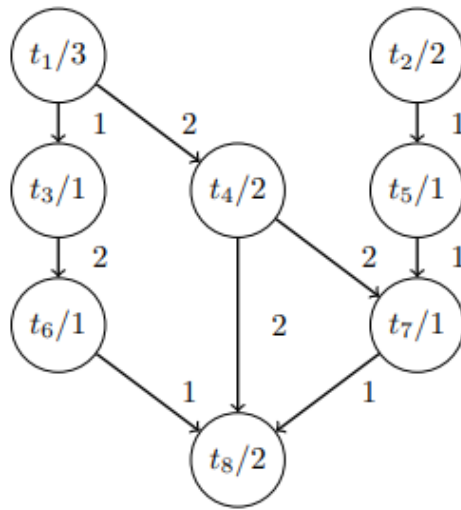


Figura 7 – Fonte: Costa (2022)

b_{ij} representa o custo de comunicação entre o processador p_i e o processador p_j .

Uma das métricas mais utilizadas no problema de escalonamento de tarefas é o *makespan*, definido como a diferença de tempo entre o início e o fim de execução de todas as tarefas do programa, ou seja, o *makespan* é o tempo total necessário para executar todas as tarefas do programa. No problema de escalonamento de tarefas o objetivo geralmente é minimizar o *makespan*, em outras palavras é executar todas as tarefas do programa no menor tempo possível. Outras métricas também podem ser utilizadas em conjunto com o *makespan*, como o balanceamento de carga entre os processadores, que visa minimizar o tempo que os processadores ficam ociosos sem executar nenhuma tarefa.

3 Trabalhos relacionados

No trabalho de [Costa \(2022\)](#) são implementados quatro algoritmos para o problema de escalonamento de tarefas em processadores heterogêneos e com custo de comunicação entre os processadores. São eles: *Heterogeneous Earliest Finish Time* ([TOPCUOGLU; HARIRI; WU, 2002](#)), *Critical Path on a Processor* ([TOPCUOGLU; HARIRI; WU, 2002](#)), *Improved Heterogeneous Earliest Finish Time* ([ALEBRAHIM; AHMAD, 2017](#)), *Improved Predict Earliest Finish* ([ZHOU et al., 2017](#)). Estes algoritmos são compostos por duas fases, onde na primeira fase é feita a priorização das tarefas e na segunda, a seleção de processadores que executará cada tarefa. Essa característica os classificam como métodos heurísticos do tipo *List-based scheduling* ([GRAHAM, 1966](#); [SHARMA, 2019](#)). No trabalho de [Costa \(2022\)](#), são usadas as métricas *makespan* e *load balance* para comparar o desempenho dos quatro algoritmos em cinco cenários distintos variando vários parâmetros do problema do escalonamento de tarefas. Este trabalho visa implementar um Algoritmo genético para o problema do escalonamento de tarefas usando o mesmo conjunto de dados e parâmetros utilizados nos cenários do trabalho de [Costa \(2022\)](#) e comparar o desempenho dos algoritmos inspirados em *List-based scheduling* com o algoritmo genético proposto. Além disso, o trabalho de [Costa \(2022\)](#) trás um algoritmo para converter grafos de tarefas de sistemas homogêneos e sem custo de comunicação para grafos de tarefas de sistemas heterogêneos e com custo de comunicação entre os diferentes processadores. Este algoritmo também será utilizado neste trabalho.

A primeira fase da implementação de um Algoritmo genético é definir sua representação, isto é, abstrair o problema escolhido e representá-lo nas estruturas de um AG, definindo seu indivíduo, operadores genéticos e sua função de avaliação (*fitness*). Para o problema do escalonamento de tarefas em ambientes multiprocessados há várias formas de representação em estruturas de um AG. O trabalho de [Silva \(2020\)](#) trás uma revisão sistemática da literatura sobre algoritmos genéticos para o problema do escalonamento de tarefas, resumindo trabalhos publicados entre 1990 e 2018. Neste trabalho foram encontradas 13 representações de indivíduos diferentes, além de 78 representação de operadores genéticos. No trabalho de [Silva \(2020\)](#) também foi comparado quatro diferentes codificações para o problema, são elas: Codificação de Listas Topológicas (TLE), Codificação de Alocação e Escalonamento (MSE), Codificação por Lista Ordenada (OLE) e Codificação por Lista de Processadores (PLE). Para a comparação foram usadas as métricas de *makespan*, *flowtime* e *load balance*. Por fim, o trabalho conclui que a representação OLE teve melhor desempenho comparada às outras, porém a representação OLE se utiliza de uma heurística de alocação de tarefas, o que pode reduzir a diversidade da população ao levar a convergência da busca para um ótimo local. As outras codificações não se dife-

riram muito em desempenho. O trabalho de [Silva \(2020\)](#) fornece uma ampla biblioteca de implementações de AGs para o problema do escalonamento de tarefas que pode ser utilizada como ponto inicial de uma pesquisa como esta monografia.

No trabalho de [Hou, Ansari e Ren \(1994\)](#) é descrito uma representação de AG para o problema do escalonamento de tarefas, definindo o indivíduo, os operadores genéticos e a função de aptidão, além de um algoritmo para a geração da população inicial de soluções. Para representar o indivíduo, usaram uma lista de processadores, onde cada processador contém uma lista das tarefas que executará. Os operadores genéticos no trabalho de [Hou, Ansari e Ren \(1994\)](#) são baseados no *crossover* de um ponto e mutação de *bit*, operadores bases muito utilizados em algoritmos genéticos. A função de aptidão usada foi o *makespan*. Ao implementar Algoritmos genéticos, podem surgir indivíduos inválidos tanto na geração da população inicial, quanto no processamento de novos indivíduos através dos operadores genéticos. No problema do escalonamento de tarefas, um indivíduo inválido pode ser aquele onde uma tarefa T_j , que depende da execução da tarefa T_i , será executada antes da execução de T_i , o que viola as restrições de ordem das tarefas. Para resolver isso, [Hou, Ansari e Ren \(1994\)](#) garantiram que seu algoritmo que gera a população inicial não gera indivíduos inválidos, essa garantia é também estendida para os algoritmos dos operadores genéticos. O trabalho de [Hou, Ansari e Ren \(1994\)](#) também compara o AG proposto com outro algoritmo para o escalonamento de tarefas baseado em lista, em cenários com grafos de tarefas gerados aleatoriamente e com grafos de programas reais. Na maioria dos cenários, o AG proposto teve um desempenho melhor do que o algoritmo baseado em lista e em nenhum cenário foi pior. O algoritmo proposto por [Hou, Ansari e Ren \(1994\)](#) considera o sistema multiprocessado homogêneo, ou seja, os processadores são considerados idênticos e executam uma mesma tarefa T_i com o mesmo custo. O AG proposto também não considera custos de comunicação entre a execução de tarefas dependentes em diferentes processadores, também chamado de *setup*. É objetivo desta monografia implementar o AG para o problema do escalonamento de tarefas em um ambiente multiprocessado heterogêneo e com custo de comunicação entre os processadores.

4 Método

Esta monografia será executada em seis etapas, são elas: estudo de implementações propostas de AGs em trabalhos correlatos, implementação do AG com base na literatura, fazer experimentos com a versão preliminar do AG, comparar os resultados com outros trabalhos relacionados, refinar o AG proposto e a escrita da monografia final.

A primeira etapa consiste no estudo de trabalhos correlatos que implementaram AGs para o problema do escalonamento de tarefas. Algumas dessas implementações não correspondem aos objetivos desta monografia, por exemplo, o algoritmo proposto por [Hou, Ansari e Ren \(1994\)](#) considera o ambiente multiprocessado como homogêneo, além de não considerar custo de comunicação entre diferentes processadores. Porém, ainda, sim, é interessante estudar sua representação do cromossomo e os operadores genéticos utilizados. Já o trabalho de [Silva \(2020\)](#) faz uma revisão sistemática da literatura de vários algoritmos genéticos para o problema do escalonamento propostos entre 1990 e 2018, compilando várias representações de cromossomo e também operadores genéticos. O trabalho de [Silva \(2020\)](#) também compara várias implementações e analisa as relações entre diversos algoritmos estudados, a fim de sintetizar e propor técnicas que demonstraram bons desempenhos para o problema investigado. Vários dos algoritmos analisados por [Silva \(2020\)](#) consideram o sistema multiprocessado heterogêneo e com custo de comunicação entre processadores, sendo estas características o objeto de estudo desta monografia.

A segunda etapa será a implementação do AG proposto por esta monografia. Como dito anteriormente, o algoritmo proposto será baseado em implementações propostas na literatura, como nos trabalhos de [Hou, Ansari e Ren \(1994\)](#), [Silva \(2020\)](#), [Santos et al. \(2023\)](#), [Omara e Arafa \(2010\)](#), [Jelodar et al. \(2006\)](#). O algoritmo será implementado em linguagem Python, usando o ambiente de desenvolvimento em nuvem *Google Colaboratory*. A entrada para o algoritmo proposto será um grafo acíclico orientado (em inglês *directed acyclic graph* ou DAG), este DAG representará o programa dividido em tarefas com seus respectivos custos e também indicando a restrição de predecessores para cada tarefa. Para a entrada do algoritmo foram utilizados DAGs disponibilizados na *Standard Task Graph* (STG), uma biblioteca de grafos mantida pela Waseda University do Japão. Porém, estes DAGs disponibilizados são para sistemas homogêneos e sem custo de comunicação. O trabalho de [Costa \(2022\)](#) propõe um algoritmo de conversão dos DAGs STG para DAGs com processadores de diferente desempenho e com custo de comunicação entre os processadores. Este algoritmo de conversão também será implementado e utilizado nesta monografia.

Após a implementação, o AG passará por uma primeira bateria de testes junto

aos DAGs de programas computacionais, visando ajustar os parâmetros como taxa de cruzamento, taxa de mutação, número de iterações, elitismo e mecanismo de seleção. Os resultados do AG serão comparados utilizando as métricas de *makespan* e *load balance*. Passada a primeira bateria de testes e definido os parâmetros, o AG proposto será submetido aos cenários descritos no trabalho de [Costa \(2022\)](#) e os resultados do AG serão comparados aos resultados do trabalho mencionado.

Após a primeira comparação de cenários, o AG passará mais uma vez por ajustes nos parâmetros a fim de melhorar ainda mais seu desempenho frente aos cenários descritos no trabalho de [Costa \(2022\)](#). Uma comparação final será feita entre os trabalhos considerando as métricas de *makespan* e *load balance*, o que resultará na discussão final sobre os resultados, finalizando com a conclusão desta monografia.

Referências

- ALEBRAHIM, S.; AHMAD, I. Task scheduling for heterogeneous computing systems. **The Journal of Supercomputing**, Springer, v. 73, p. 2313–2338, 2017. Citado na página 15.
- BITTENCOURT, L. F.; GOLDMAN, A.; MADEIRA, E. R.; FONSECA, N. L. da; SAKELLARIOU, R. Scheduling in distributed systems: A cloud computing perspective. **Computer science review**, Elsevier, v. 30, p. 31–54, 2018. Citado na página 12.
- COELLO, C. A.; LAMONT, G. B.; VELDHUIZEN, D. A. V. **Evolutionary Algorithms for Solving Multi-Objective Problems**. Springer New York, NY, 2007. Disponível em: <<https://link.springer.com/book/10.1007/978-0-387-36797-2>>. Citado na página 8.
- COSTA, B. C. S. **Avaliação de algoritmos de escalonamento de aplicações paralelas em processadores heterogêneos**. Uberlândia: [s.n.], 2022. 56 p. Citado 7 vezes nas páginas 2, 5, 6, 14, 15, 17 e 18.
- EIBEN, A.; SMITH, J. **Introduction to Evolutionary Computing**. Springer Berlin, Heidelberg, 2015. Disponível em: <<https://link.springer.com/book/10.1007/978-3-662-44874-8>>. Citado na página 11.
- Gabriela Pederneiras. **Cloud, ou computação em nuvem, na indústria 4.0**. 2019. Disponível em: <<https://www.industria40.ind.br/artigo/17984-cloud-ou-computacao-em-nuvem-na-industria-40>>. Acesso em: 26 de fevereiro 2024. Citado na página 5.
- GOLDBERG, D. E.; DEB, K. A comparative analysis of selection schemes used in genetic algorithms. In: RAWLINS, G. J. (Ed.). Elsevier, 1991, (Foundations of Genetic Algorithms, v. 1). p. 69–93. Disponível em: <<https://www.sciencedirect.com/science/article/pii/B9780080506845500082>>. Citado na página 9.
- GOLUB, M.; KASAPOVIC, S. Scheduling multiprocessor tasks with genetic algorithms. In: CITESEER. **APPLIED INFORMATICS-PROCEEDINGS-**. [S.l.], 2002. p. 273–278. Citado na página 11.
- GRAHAM, R. L. Bounds for certain multiprocessing anomalies. **Bell system technical journal**, Wiley Online Library, v. 45, n. 9, p. 1563–1581, 1966. Citado na página 15.
- HOLLAND, J. **Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence**. Oxford, England: U Michigan Press, 1975. Citado 3 vezes nas páginas 7, 11 e 12.
- HOU, E. S.; ANSARI, N.; REN, H. A genetic algorithm for multiprocessor scheduling. **IEEE Transactions on Parallel and Distributed systems**, IEEE, v. 5, n. 2, p. 113–120, 1994. Citado 2 vezes nas páginas 16 e 17.
- JELODAR, M.; FAKHRAIE, S.; MONTAZERI, F.; FAKHRAIE, S.; AHMADABADI, M. A representation for genetic-algorithm-based multiprocessor task scheduling. In:

2006 IEEE International Conference on Evolutionary Computation. [s.n.], 2006. p. 340–347. Disponível em: <<https://ieeexplore.ieee.org/document/1688328>>. Citado 2 vezes nas páginas 8 e 17.

OMARA, F.; ARAFA, M. Genetic algorithms for task scheduling problem. **J. Parallel Distrib. Comput.**, v. 70, p. 13–22, 01 2010. Citado 2 vezes nas páginas 6 e 17.

PADUA, D. **Encyclopedia of parallel computing.** [S.l.]: Springer Science & Business Media, 2011. Citado na página 13.

SANTOS, J. F. et al. Algoritmos evolutivos multiobjetivo baseados em tabelas para escalonamento de tarefas em ambientes multiprocessados. Universidade Federal de Uberlândia, 2023. Citado 2 vezes nas páginas 6 e 17.

SANTOS, S. **Computação de alto desempenho: essencial na vida diária e o futuro da tecnologia.** 2021. Disponível em: <<https://canaltech.com.br/colunas/computacao-de-alto-desempenho-essencial-na-vida-diaria-e-o-futuro-da-tecnologia/>>. Citado na página 5.

SHARMA, N. **Static list scheduling for DAGs: A comparative study between algorithms.** 2019. Acesso em: 12 abril 2024. Disponível em: <https://github.com/sharma-n/DAG_Scheduling>. Citado na página 15.

SILVA, E. C. da. **Representações de Algoritmos Genéticos para o Problema de Escalonamento Estático de Tarefas em Multiprocessadores.** Uberlândia: [s.n.], 2020. 231 p. Citado 10 vezes nas páginas 2, 5, 8, 9, 10, 11, 12, 15, 16 e 17.

TOPCUOGLU, H.; HARIRI, S.; WU, M.-Y. Performance-effective and low-complexity task scheduling for heterogeneous computing. **IEEE transactions on parallel and distributed systems**, IEEE, v. 13, n. 3, p. 260–274, 2002. Citado na página 15.

ZHOU, N.; QI, D.; WANG, X.; ZHENG, Z.; LIN, W. A list scheduling algorithm for heterogeneous systems based on a critical node cost table and pessimistic cost table. **Concurrency and Computation: Practice and Experience**, Wiley Online Library, v. 29, n. 5, p. e3944, 2017. Citado na página 15.

ZOMAYA, A.; WARD, C.; MACEY, B. Genetic scheduling for parallel processor systems: comparative studies and performance issues. **IEEE Transactions on Parallel and Distributed Systems**, v. 10, n. 8, p. 795–812, 1999. Citado na página 11.